

Migración de clasificación IA: de Gemini a Qwen local

Qualitrack | Informe técnico | 10 de septiembre de 2026

1. Dónde estás hoy

Revisé el código antes de responder, porque el punto de partida cambia bastante la recomendación. Ya tienes IA local a medias:

- `backend/embedding-service/app.py` corre **Qwen3-Embedding-0.6B** en CPU (FastAPI, endpoint `/embed`, 768 dimensiones truncadas), y `vector.service.js` guarda los chunks en pgvector.
- `test-classifier-qwen.js` ya prueba `classifyDocumentByEmbeddings`, es decir, el ranking semántico local sobre los subcriterios 9.1 a 9.5.
- Lo único que sigue en Gemini es `classifier.service.js:83` — `gemini-2.5-flash` con `responseSchema`, que decide relevancia, justificación y `evidenceFragment`.

Así que no es *migrar todo a Qwen*: es **reemplazar una sola función** por un LLM local. Eso simplifica muchísimo el problema.

2. Qué modelo para tu PC

Tu RTX 4070 SUPER tiene 12 GB de VRAM y ~504 GB/s de ancho de banda de memoria. Eso lo define todo:

Modelo	Peso Q4	¿Cabe en 12 GB?	Veredicto
Qwen3.5-4B	3,4 GB	Sobrado	Suficiente para clasificar; ideal si quieres concurrencia
Qwen3.5-9B	6,6 GB	Sí, con holgura	Recomendado
Qwen3.6-27B	17 GB	No	Offload a RAM → ~3-5 tok/s, inusable

Qwen3.5-9B es el punto óptimo: pesa 6,6 GB en Q4_K_M, deja unos 4-5 GB libres para el KV cache (te alcanza para 32K de contexto de sobra, considerando que tu prompt trunca a 12.000 caracteres ≈ 4.000 tokens), y en benchmarks de conocimiento e instruction-following pelea contra modelos diez veces más grandes. Contexto nativo de 256K y además acepta imágenes, lo que a futuro te sirve para PDFs escaneados sin OCR.

Espera unos **45-60 tok/s**. Una respuesta JSON de tu clasificador son ~150 tokens, o sea **2 a 4 segundos por documento**, comparable a Gemini Flash.

Si al evaluar ves que el 9B alucina códigos de subcriterio, la solución no es subir de tamaño: es reducirle el trabajo (ver sección 4).

3. Instalación en tu PC

Ollama sobre Windows es la opción correcta acá: CUDA nativo sin WSL, endpoint compatible con OpenAI, y soporta format con JSON Schema, que es el equivalente directo a tu `responseSchema` de Gemini. vLLM da unas 3x de throughput, pero exige Linux o WSL2 y cuantización AWQ/GPTQ: no vale la

pena para un solo usuario.

Instalación y descarga del modelo

```
winget install Ollama.Ollama  
  
ollama pull qwen3.5:9b
```

Variables de entorno

En System → Environment Variables, reiniciando Ollama después:

- OLLAMA_HOST=0.0.0.0:11434 — para que el backend en Docker lo alcance.
- OLLAMA_KEEP_ALIVE=-1 — mantiene el modelo en VRAM y evita 15 s de carga en cada request.
- OLLAMA_CONTEXT_LENGTH=16384

Prueba de humo con schema forzado

```
curl http://localhost:11434/api/chat -d '{  
  "model": "qwen3.5:9b",  
  "stream": false,  
  "format": {  
    "type": "object",  
    "properties": {  
      "relevant": { "type": "boolean" },  
      "subcriterionCode": { "type": ["string", "null"] },  
      "confidence": { "type": "number" },  
      "justification": { "type": "string" }  
    },  
    "required": ["relevant", "confidence", "justification"]  
  },  
  "messages": [{ "role": "user", "content":  
    "El comite de calidad realiza autoevaluacion periodica. Clasifica en 9.1-9.5." }]  
}'
```

4. Cómo integrarlo al código

No toques `classifyByAI` directamente. Mete una capa de proveedor:

- Nuevo **backend/src/services/llm.service.js**: cliente HTTP contra `process.env.LLM_SERVICE_URL` (`http://host.docker.internal:11434` en local), traduciendo tu `RESPONSE_SCHEMA` al format de Ollama.
- En `classifyText`, cambia el `if` (`process.env.GEMINI_API_KEY`) por un `switch` sobre `CLASSIFIER_PROVIDER=local|gemini|keywords`, manteniendo intacto el fallback a `keywords`.
- Agrega el servicio al **docker-compose.yml** igual que `embedding-service`, o apunta directamente a la URL del túnel.

El truco de calidad. Hoy le pasas los cinco subcriterios a Gemini y le pides que elija. Con un 9B local conviene usar el ranking de embeddings que ya tienes y pasarle **solo el top-3** de semanticRanking, convirtiendo la tarea de «clasificar» en «verificar y justificar». Un 9B hace eso casi tan bien como Flash. Extiende `test-classifier-qwen.js` para correr los 8 casos contra ambos proveedores y comparar accuracy antes de cortar Gemini.

5. Conectarlo a producción: acá está el problema real

Tu producción es Railway (`railway.json`, `Dockerfile single-service`). **Railway no tiene GPU**, así que el modelo no puede vivir ahí. Opciones, de peor a mejor:

A) Tu PC como servidor de inferencia vía túnel

Cloudflare Tunnel o Tailscale desde tu casa hacia Railway. Funciona para demo o tesis y cuesta \$0.

```
cloudflared tunnel --url http://localhost:11434
```

Advertencia importante: Ollama no tiene autenticación. Nunca abras el puerto 11434 por port-forwarding del router. Pon Caddy o nginx delante exigiendo un `Authorization: Bearer`, o usa Cloudflare Access con service token. Y asume que tu PC debe estar encendido 24/7, con la latencia de tu ISP.

B) Servidor GPU propio o de la universidad

Es la única opción que sostiene de verdad el argumento de privacidad: los documentos de acreditación nunca salen de la institución. Si el proyecto es para una universidad, esta es la conversación que vale la pena tener.

C) VPS con GPU

Hetzner, Scaleway L4, RunPod. Los pesos son tuyos, no hay Google en el medio, y corres vLLM con buen throughput. Los datos salen de tu red, pero bajo tu control contractual. Es el punto medio realista si no hay hardware institucional.

D) CPU en Railway con Qwen3.5-2B

Técnicamente posible (ya lo haces con el embedding service), pero 2-5 tok/s significan 30 a 60 segundos por documento. Solo como último recurso.

Recomendación

Implementa contra `LLM_SERVICE_URL` para que A, B y C sean un simple cambio de variable de entorno. Desarrolla con Ollama local, demuestra con Cloudflare Tunnel, y deja documentado B como el destino de producción.

Fuentes

- Ollama — biblioteca qwen3.5 — <https://ollama.com/library/qwen3.5>
- Qwen en Hugging Face — <https://huggingface.co/Qwen>
- Artificial Analysis — Qwen3.5 9B — <https://artificialanalysis.ai/models/qwen3-5-9b>
- vLLM vs Ollama 2026 — <https://tech-insider.org/vllm-vs-ollama-2026/>

- Structured output con Ollama
<https://www.glukhov.org/llm-performance/ollama/llm-structured-output-with-ollama-in-python-and-go/>